this diagram is one of the reasons I recommend using Razor Pages wherever possible!

TIP You can modify all these conventions, including the algorithm shown in figure 19.8, during initial configuration. In fact, you can replace the whole Razor templating engine if you really want to!

You may find it tempting to explicitly provide the name of the view file you want to render in your controller; if so, I'd encourage you to fight that urge. You'll have a much simpler time if you embrace the conventions as they are and go with the flow. That extends to anyone else who looks at your code; if you stick to the standard conventions, there'll be a comforting familiarity when they look at your app. That can only be a good thing!

As well as providing a view template name, you can also pass an object to act as the view model for the Razor view. This object should match the type specified in the view's `@model` directive, and it's accessed in exactly the same way as for Razor Pages; using the `Model` property.

TIP All the other ways of passing data to the view I described in chapter 17 are available in MVC controllers too. You should generally favor the view model where possible, but you can also use `ViewData`, `TempData`, or `@inject` services, for example.

The following listing shows two examples of passing a view model to a view.

Listing 19.7 Returning `ViewResult` from an action method using default conventions

```
public class ToDoController : Controller
{
    public IActionResult Index()
    {
```